Graphical User Interfaces (GUIs) JavaFX GUI Basics

CSE 114: Introduction to Object-Oriented Programming

Paul Fodor

Stony Brook University

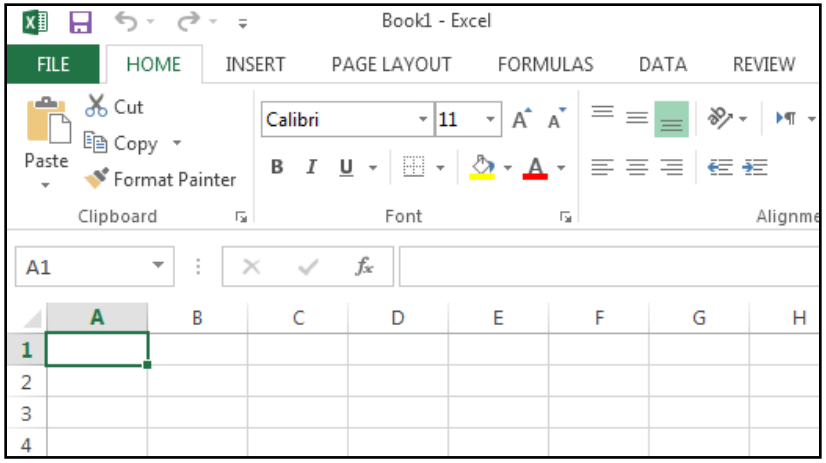
http://www.cs.stonybrook.edu/~cse114

Contents

- GUI Examples
- Graphical User Interfaces (GUIs)
- What does a GUI framework do for you?
- JavaFX vs AWT and Swing
- How do GUIs work?
- GUI Look vs. Behavior
- Basic Structure of a JavaFX GUI
- Layout Panes, UI Controls, and Shapes
- Display Shapes
- Helper classes: The Color and Font Classes
- Line, Rectangle, Circle, Ellipse, Arc, Polygon and Polyline
- The Image and ImageView Classes
- JavaFX CSS styles

(c) Paul Fodor and Pearson Inc.

GUI Examples









(c) Paul Fodor and Pearson Inc.

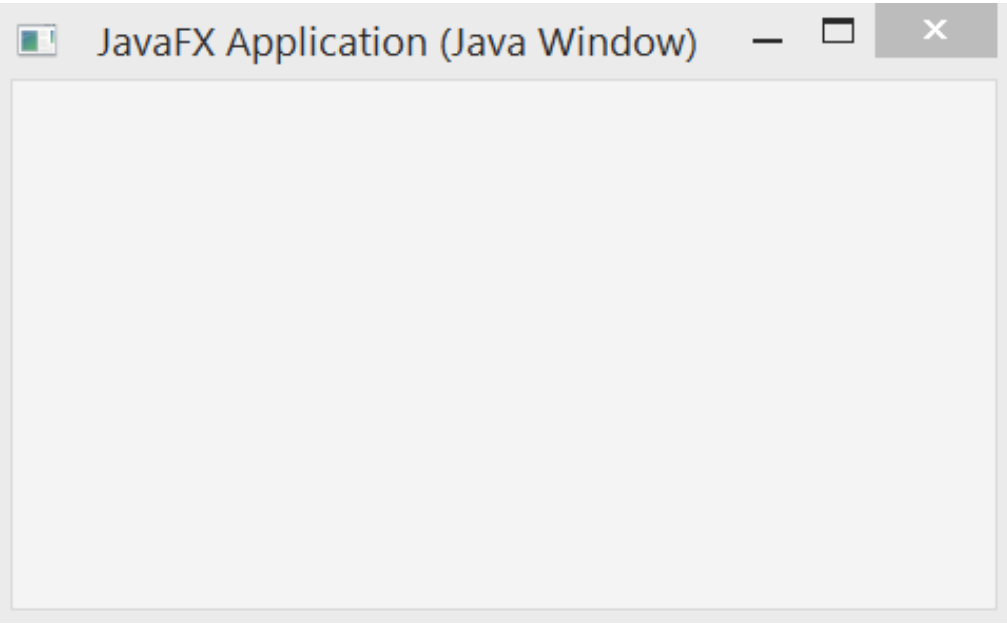
Graphical User Interfaces (GUIs)

- Graphical User Interfaces (GUIs)
  - provides user-friendly human interaction
  - they were the initial motivation for object-oriented programming
    - predefined classes for GUI components, event processing interfaces
- Building GUIs require use of GUI frameworks:
  - JavaFX (part of JSE 8, 2014)
  - Older Java frameworks:
    - Abstract Window Toolkit (AWT):
      - **`java.awt.*, java.awt.geom.*, java.awt.event.*`**
    - SWING:
      - **`javax.swing.*, javax.swing.event.*`**

(c) Paul Fodor and Pearson Inc.

What does a GUI framework do for you?

- Provides ready, interactive, customizable components and event handling management
  - you wouldn't want to have to code your own window class, label class, text fields and text areas classes, button class, checkbox, radio buttons, lists, geometrical figures, containers/layout managers, etc.



(c) Paul Fodor and Pearson Inc.

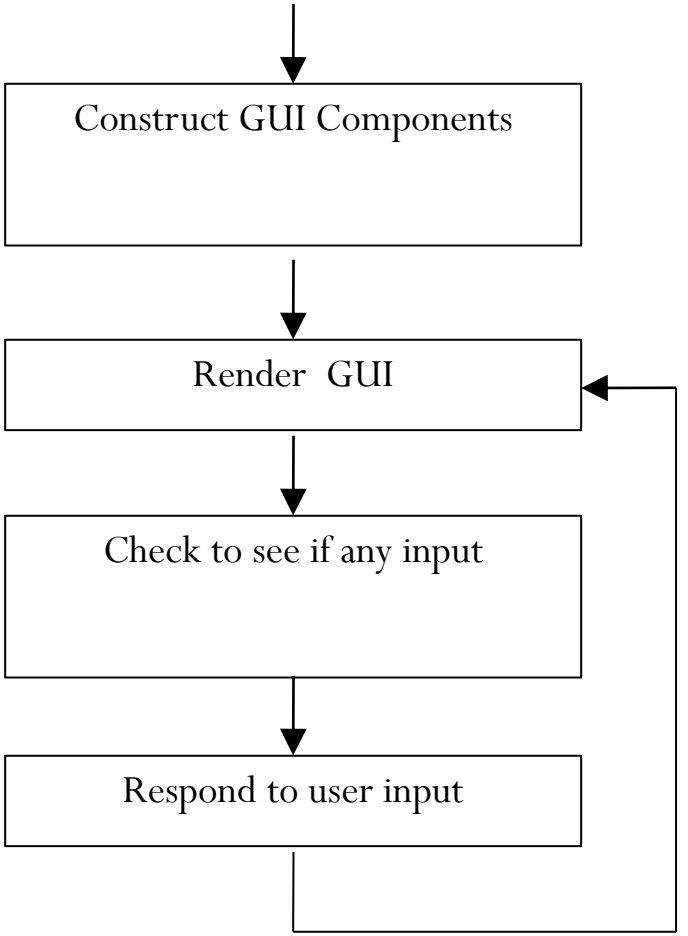
JavaFX vs AWT and Swing

- Swing and AWT were older Java frameworks replaced by the JavaFX platform for developing rich Internet applications in JDK8.
  - When Java was introduced, the GUI classes were bundled in a library known as the Abstract Windows Toolkit (AWT).
    - AWT was prone to platform-specific bugs
    - AWT was fine for developing simple graphical user interfaces, but not for developing comprehensive GUI projects.
  - The AWT user-interface components were replaced by a more robust, versatile, and flexible library known as Swing components.
    - Swing components were painted directly on canvases using Java code.
    - Swing components depended less on the target platform and used less of the native GUI resource.
  - **With the release of Java 8, Swing was replaced by a completely new GUI platform: JavaFX.**

(c) Paul Fodor and Pearson Inc.

How do GUIs work?

- GUIs loop and respond to events:
  - Example: a mouse click on a button
    - The Operating System recognizes mouse click, determines which window it was inside and notifies that program by putting the event on that program's input buffer/queue
    - The program runs in loop:
      - renders the GUI
      - checks input buffer filled by OS
      - if it finds a mouse click, determines which component in the program, respond appropriately according to handler

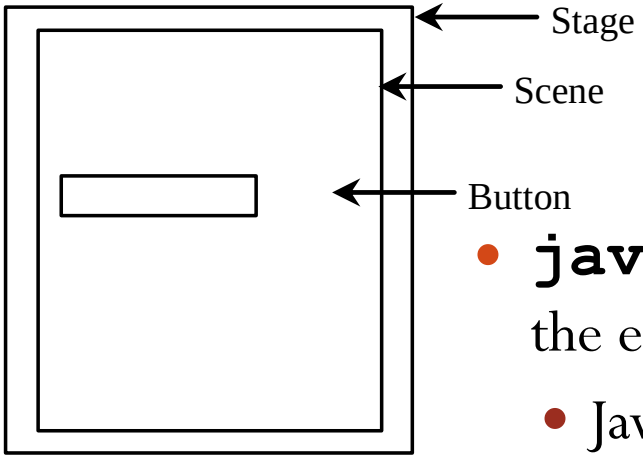


(c) Paul Fodor and Pearson Inc.

GUI Look vs. Behavior

- Look: physical appearance
  - GUIs components
  - containment and layout management
- Behavior: responding to events
  - event programmed response through event handlers that we define

(c) Paul Fodor and Pearson Inc.



Basic Structure of a JavaFX GUI

- **`javafx.application.Application`** is the entry point for JavaFX applications
  - JavaFX creates an application thread for running the application start method, processing input events, and running animation timelines.
  - We override the **`start(Stage)`** method!
- **`javafx.stage.Stage`** is the top level JavaFX container (i.e., window)
  - The primary Stage is constructed by the platform.
- **`javafx.scene.Scene`** class is the container for all content in a scene graph in the stage.
- **`javafx.scene.Node`** is the base class for scene graph nodes (i.e., components).

(c) Paul Fodor and Pearson Inc.

```
import javafx.application.Application;
import javafx.stage.Stage;
import javafx.scene.Scene;
import javafx.scene.control.Button;

public class MyFirstJavaFX extends Application {
  @Override // Override the start method in the Application class
  public void start(Stage primaryStage) {
    // Create a button and place it in the scene
    Button btOK = new Button("OK");
    Scene scene = new Scene(btOK, 200, 250);
    primaryStage.setScene(scene); // Place the scene in the stage
    primaryStage.setTitle("MyJavaFX"); // Set the stage title
    primaryStage.show(); // Display the stage
  }

  public static void main(String[] args) {
    launch(args);
  }
}
```



(c) Paul Fodor and Pearson Inc.

```
// Multiple stages can be added beside the primaryStage
import javafx.application.Application;
import javafx.stage.Stage;
import javafx.scene.Scene;
import javafx.scene.control.Button;

public class MultipleStageDemo extends Application {
  @Override // Override the start method in the Application class
  public void start(Stage primaryStage) {
    // Create a scene and place a button in the scene
    Scene scene = new Scene(new Button("OK"), 200, 250);
    primaryStage.setTitle("MyJavaFX"); // Set the stage title
    primaryStage.setScene(scene); // Place the scene in the stage
    primaryStage.show(); // Display the stage
    Stage stage = new Stage(); // Create a new stage
    stage.setTitle("Second Stage"); // Set the stage title
    // Set a scene with a button in the stage
    stage.setScene(new Scene(new Button("New Stage"), 100, 100));
    stage.show(); // Display the stage
  }

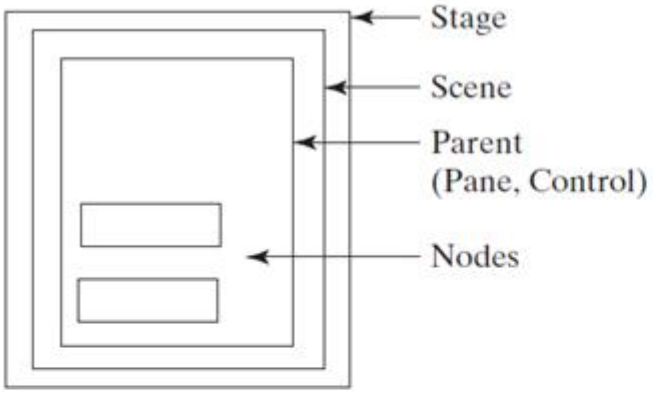
  public static void main(String[] args) {
    launch(args);
  }
}
```

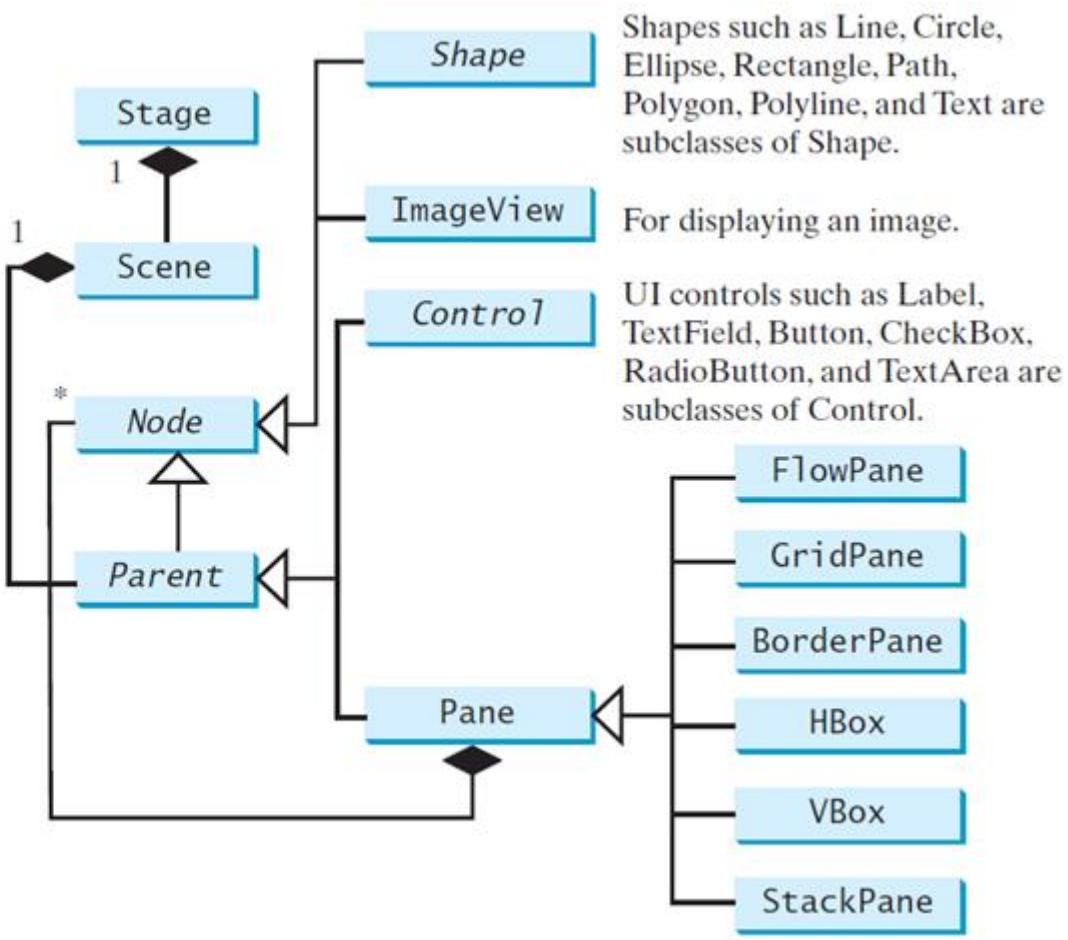




(c) Paul Fodor and Pearson Inc.

Panes, UI Controls, and Shapes





(c) Paul Fodor and Pearson Inc.

Layout Panes

- JavaFX provides many **types of panes for organizing nodes in a container.**

Class	*Description*
`Pane`	Base class for layout panes. It contains the `getChildren()` method for returning a list of nodes in the pane.
`StackPane`	Places the nodes on top of each other in the center of the pane.
`FlowPane`	Places the nodes row-by-row horizontally or column-by-column vertically.
`GridPane`	Places the nodes in the cells in a two-dimensional grid.
`BorderPane`	Places the nodes in the top, right, bottom, left, and center regions.
`HBox`	Places the nodes in a single row.
`VBox`	Places the nodes in a single column.

(c) Paul Fodor and Pearson Inc.

```
import javafx.application.Application;
import javafx.stage.Stage;
import javafx.scene.Scene;
import javafx.scene.layout.StackPane;
import javafx.scene.control.Button;

public class ButtonInPane extends Application {

  @Override // Override the start method in the Application class
  public void start(Stage primaryStage) {
    // Create a scene and place a button in the scene
    StackPane pane = new StackPane();
    pane.getChildren().add(new Button("OK"));
    Scene scene = new Scene(pane, 200, 50);
    primaryStage.setTitle("Button in a pane"); // Set the stage title
    primaryStage.setScene(scene); // Place the scene in the stage
    primaryStage.show(); // Display the stage
  }

  public static void main(String[] args) {
    launch(args);
  }
}
```



(c) Paul Fodor and Pearson Inc.

FlowPane

The getter and setter methods for property values and a getter for property itself are provided in the class, but omitted in the UML diagram for brevity.

javafx.scene.layout.FlowPane

```
-alignment: ObjectProperty<Pos>
-orientation:
   ObjectProperty<Orientation>
-hgap: DoubleProperty
-vgap: DoubleProperty
```

```
+FlowPane()
+FlowPane(hgap: double, vgap:
   double)
+FlowPane(orientation:
   ObjectProperty<Orientation>)
+FlowPane(orientation:
   ObjectProperty<Orientation>,
   hgap: double, vgap: double
```

The overall alignment of the content in this pane (default: `Pos.LEFT`).

The orientation in this pane (default: `Orientation.HORIZONTAL`).

The horizontal gap between the nodes (default: `0`).

The vertical gap between the nodes (default: `0`).

Creates a default `FlowPane`.

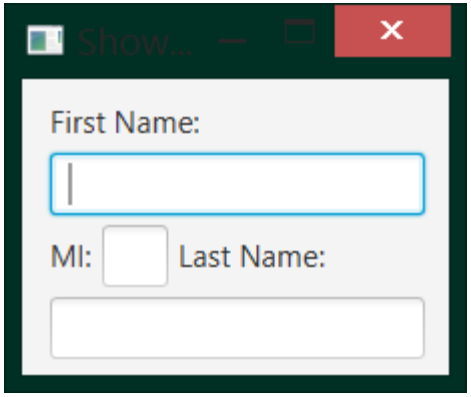
Creates a `FlowPane` with a specified horizontal and vertical gap.

Creates a `FlowPane` with a specified orientation.

Creates a `FlowPane` with a specified orientation, horizontal gap and vertical gap.

(c) Paul Fodor and Pearson Inc.

```
import javafx.application.Application;
import javafx.stage.Stage;
import javafx.scene.Scene;
import javafx.scene.layout.FlowPane;
import javafx.scene.control.Label;
import javafx.scene.control.TextField;
import javafx.geometry.Insets;
public class ShowFlowPane extends Application {
  @Override
  public void start(Stage primaryStage) {
    FlowPane pane = new FlowPane();
    pane.setPadding(new Insets(11, 12, 13, 14));
    pane.setHgap(5);
    pane.setVgap(5);
    // Place nodes in the pane
    pane.getChildren().addAll(new Label("First Name:"),
      new TextField(), new Label("MI:"));
    TextField tfMi = new TextField();
    tfMi.setPrefColumnCount(1);
    pane.getChildren().addAll(tfMi, new Label("Last Name:"),
      new TextField());
    // Create a scene and place it in the stage
    Scene scene = new Scene(pane, 210, 150);
    primaryStage.setTitle("ShowFlowPane");
    primaryStage.setScene(scene); // Place the scene in the stage
    primaryStage.show(); // Display the stage
  }
  public static void main(String[] args) {
    launch(args);
  }
}
```



(c) Paul Fodor and Pearson Inc.

GridPane

The getter and setter methods for property values and a getter for property itself are provided in the class, but omitted in the UML diagram for brevity.

javafx.scene.layout.GridPane

```
-alignment: ObjectProperty<Pos>
-gridLinesVisible:
   BooleanProperty
-hgap: DoubleProperty
-vgap: DoubleProperty
```

```
+GridPane()
+add(child: Node, columnIndex:
   int, rowIndex: int): void
+addColumn(columnIndex: int,
   children: Node...): void
+addRow(rowIndex: int,
   children: Node...): void
+getColumnIndex(child: Node):
   int
+setColumnIndex(child: Node,
   columnIndex: int): void
+getRowIndex(child:Node): int
+setRowIndex(child: Node,
   rowIndex: int): void
+setHalighnment(child: Node,
   value: HPos): void
+setValighnment(child: Node,
   value: VPos): void
```

The overall alignment of the content in this pane (default: `Pos.LEFT`).
Is the grid line visible? (default: `false`)

The horizontal gap between the nodes (default: 0).
The vertical gap between the nodes (default: 0).

Creates a `GridPane`.
Adds a node to the specified column and row.

Adds multiple nodes to the specified column.

Adds multiple nodes to the specified row.

Returns the column index for the specified node.

Sets a node to a new column. This method repositions the node.

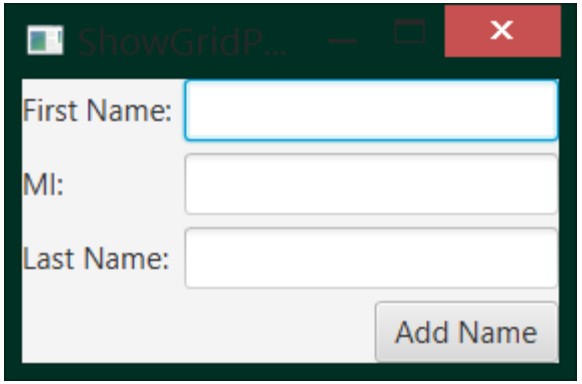
Returns the row index for the specified node.
Sets a node to a new row. This method repositions the node.

Sets the horizontal alignment for the child in the cell.

Sets the vertical alignment for the child in the cell.

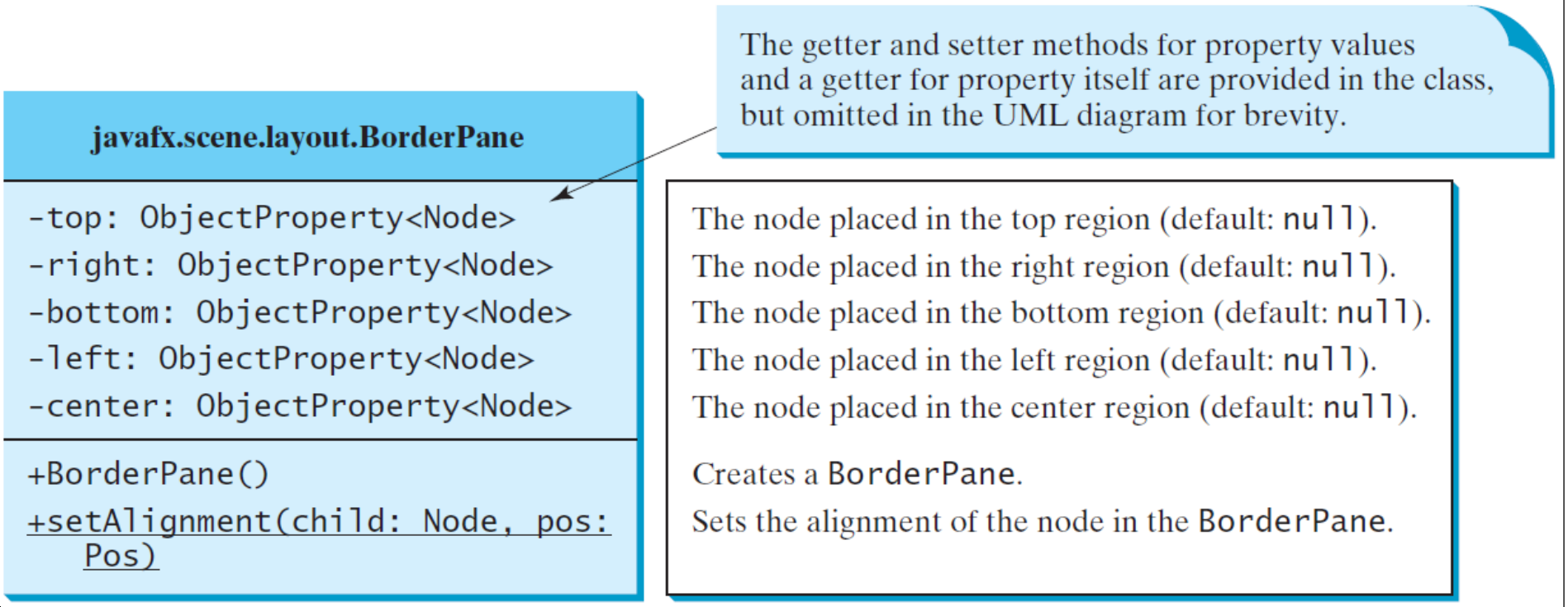
(c) Paul Fodor and Pearson Inc.

```
import javafx.application.Application;
import javafx.stage.Stage;
import javafx.scene.Scene;
import javafx.scene.layout.GridPane;
import javafx.scene.control.*;
import javafx.geometry.*;
public class ShowGridPane extends Application {
  @Override
  public void start(Stage primaryStage) {
    // Create a pane and set its properties
    GridPane pane = new GridPane();
    pane.setAlignment(Pos.CENTER);
    pane.setHgap(5.5);
    pane.setVgap(5.5);
    // Place nodes in the pane at positions   column,row
    pane.add(new Label("First Name:"), 0, 0);
    pane.add(new TextField(), 1, 0);
    pane.add(new Label("MI:"), 0, 1);
    pane.add(new TextField(), 1, 1);
    pane.add(new Label("Last Name:"), 0, 2);
    pane.add(new TextField(), 1, 2);
    Button btAdd = new Button("Add Name");
    pane.add(btAdd, 1, 3);
    GridPane.setHalignment(btAdd, HPos.RIGHT);
    // Create a scene and place it in the stage
    Scene scene = new Scene(pane);
    primaryStage.setTitle("ShowGridPane");
    primaryStage.setScene(scene); primaryStage.show(); }
  public static void main(String[] args) {
    launch(args);
  }}
```



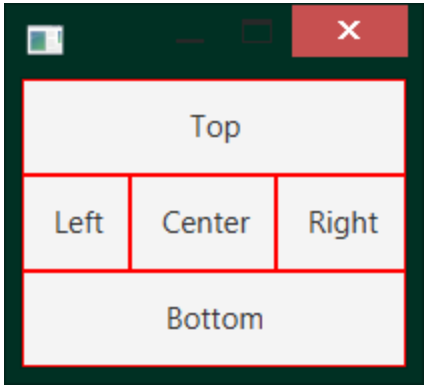
(c) Paul Fodor and Pearson Inc.

BorderPane



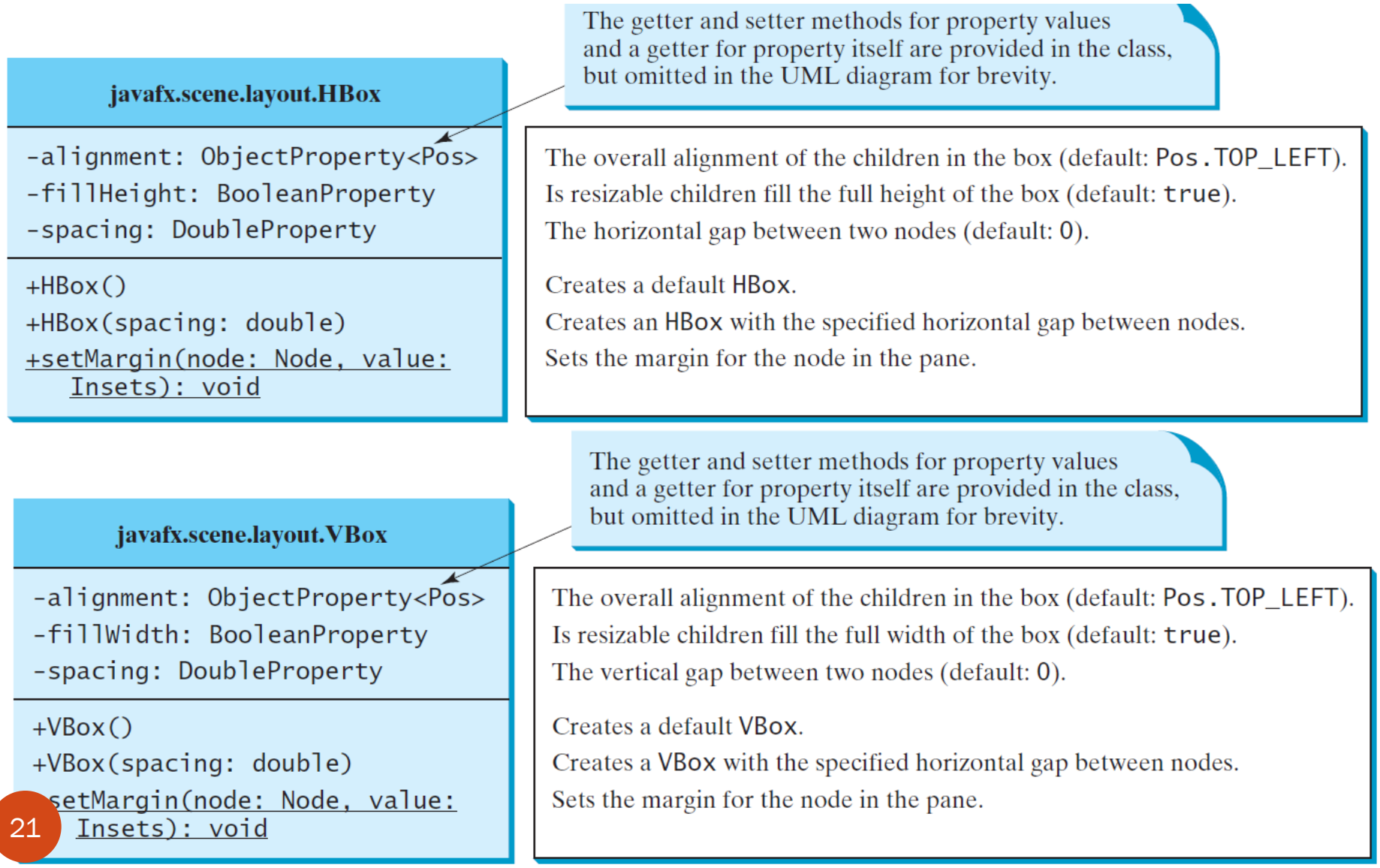
(c) Paul Fodor and Pearson Inc.

```
import javafx.application.Application;
import javafx.stage.Stage;
import javafx.scene.Scene;
import javafx.scene.layout.BorderPane;
import javafx.scene.layout.StackPane;
import javafx.scene.control.Label;
import javafx.geometry.Insets;
public class ShowBorderPane extends Application {
  @Override
  public void start(Stage primaryStage) {
    BorderPane pane = new BorderPane();
    pane.setTop(new CustomPane("Top"));
    pane.setRight(new CustomPane("Right"));
    pane.setBottom(new CustomPane("Bottom"));
    pane.setLeft(new CustomPane("Left"));
    pane.setCenter(new CustomPane("Center"));
    Scene scene = new Scene(pane);
    primaryStage.setScene(scene); primaryStage.show();
  }
  public static void main(String[] args) {
    launch(args);
  }
}
class CustomPane extends StackPane {
  public CustomPane(String title) {
    getChildren().add(new Label(title));
    setStyle("-fx-border-color: red");
    setPadding(new Insets(11.5, 12.5, 13.5, 14.5));
  }
}
```

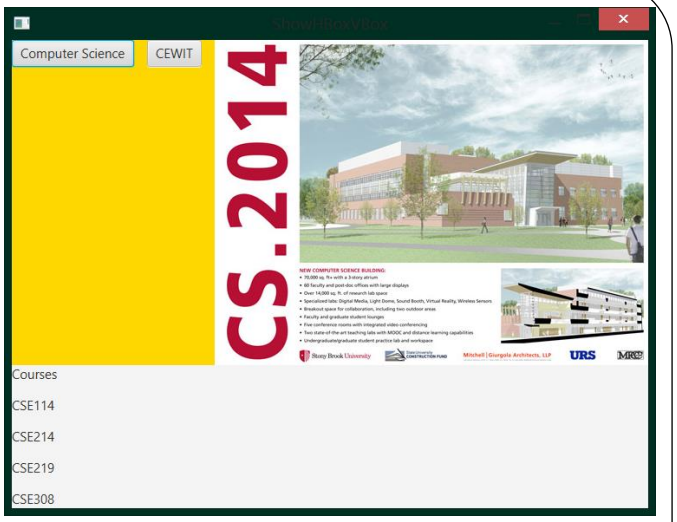


(c) Paul Fodor and Pearson Inc.

Hbox and VBox

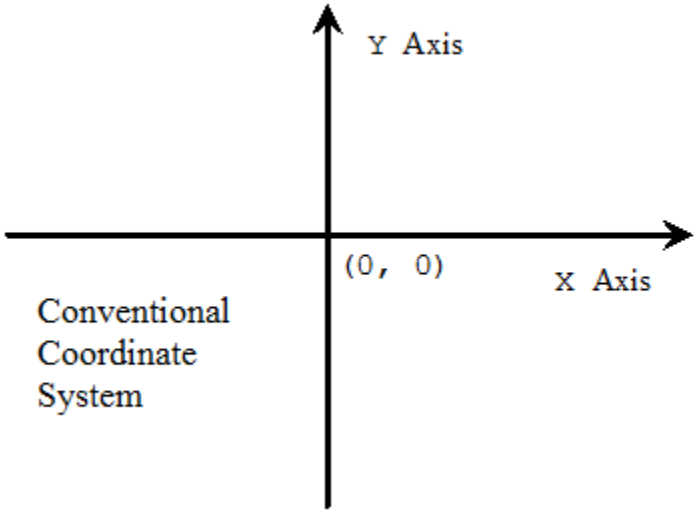


```
import javafx.application.Application;
import javafx.stage.Stage;
import javafx.scene.Scene;
import javafx.scene.layout.BorderPane;
import javafx.scene.layout.HBox;
import javafx.scene.layout.VBox;
import javafx.scene.control.Button;
import javafx.scene.control.Label;
import javafx.scene.image.Image;
import javafx.scene.image.ImageView;
public class ShowHBoxVBox extends Application {
  @Override
  public void start(Stage primaryStage) {
    BorderPane pane = new BorderPane();
    HBox hBox = new HBox(15);
    hBox.setStyle("-fx-background-color: gold");
    hBox.getChildren().add(new Button("Computer Science"));
    hBox.getChildren().add(new Button("CEWIT"));
    ImageView imageView = new ImageView(new Image("cs14.jpg"));
    hBox.getChildren().add(imageView);
    pane.setTop(hBox);
    VBox vBox = new VBox(15);
    vBox.getChildren().add(new Label("Courses"));
    Label[] courses = {new Label("CSE114"), new Label("CSE214"),
        new Label("CSE219"), new Label("CSE308")};
    for (Label course: courses) {
      vBox.getChildren().add(course);
    }
    pane.setLeft(vBox);
    Scene scene = new Scene(pane); primaryStage.setScene(scene);
    primaryStage.show();
  }
```

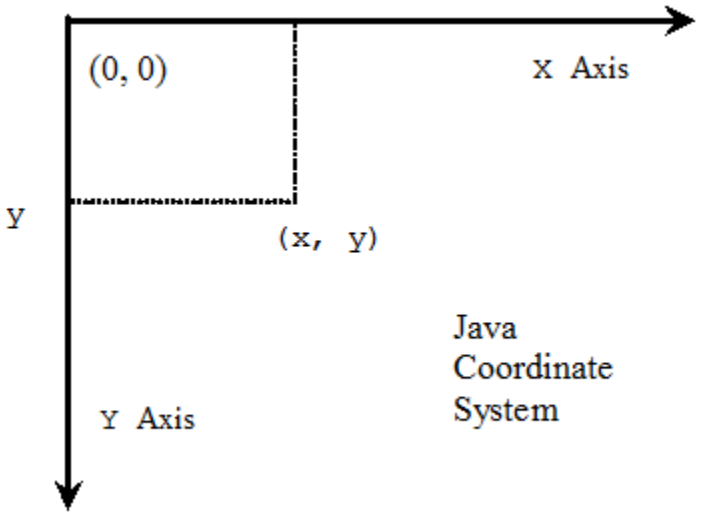


(c) Paul Fodor and Pearson Inc.

Display Shapes

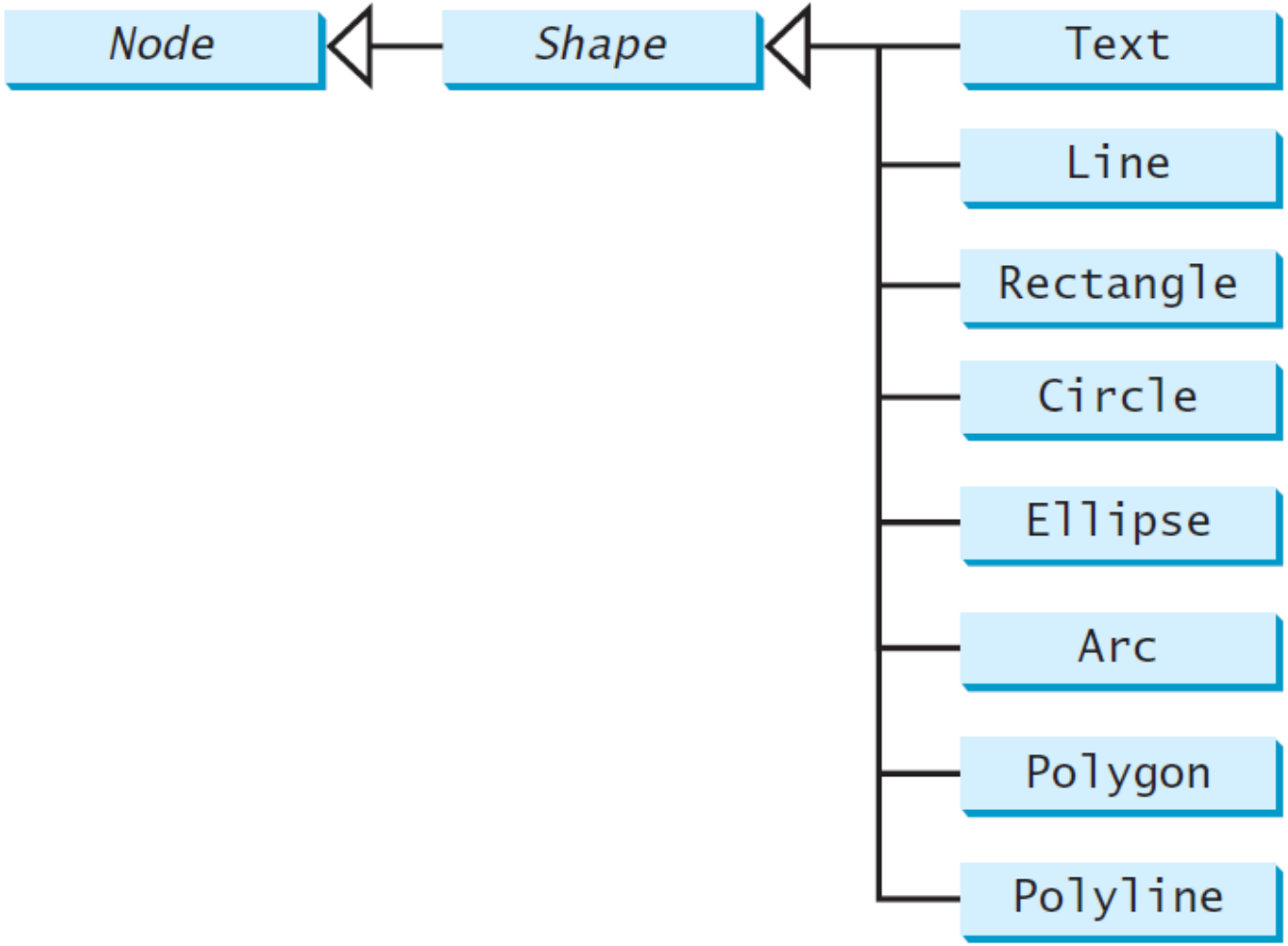


- Programming Coordinate Systems start from the left-upper corner

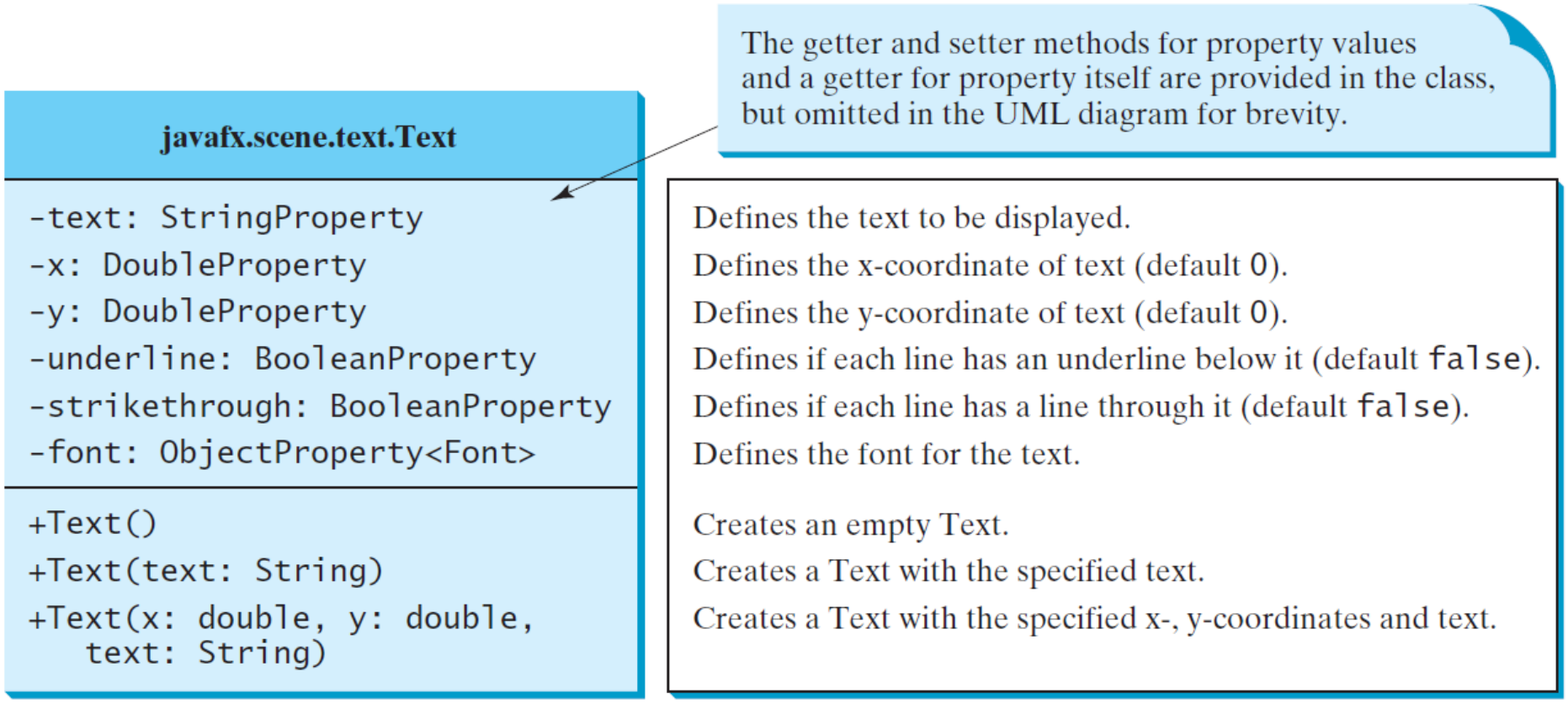


Shapes

JavaFX provides many shape classes for drawing texts, lines, circles, rectangles, ellipses, arcs, polygons, and polylines.

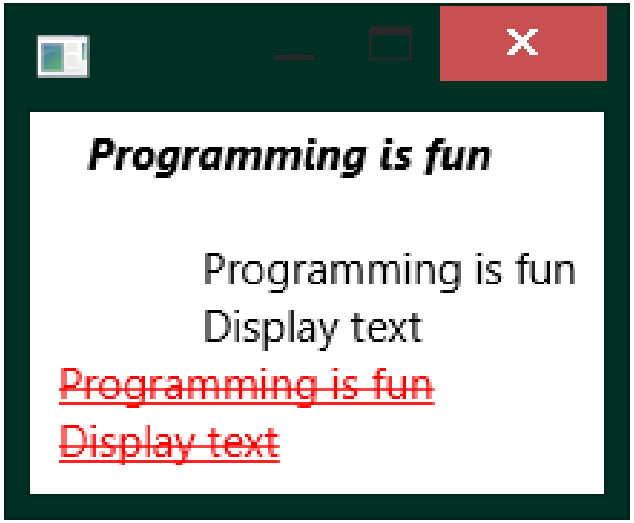


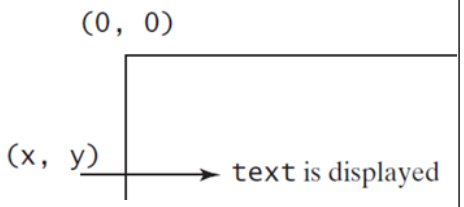
Text



(c) Paul Fodor and Pearson Inc.

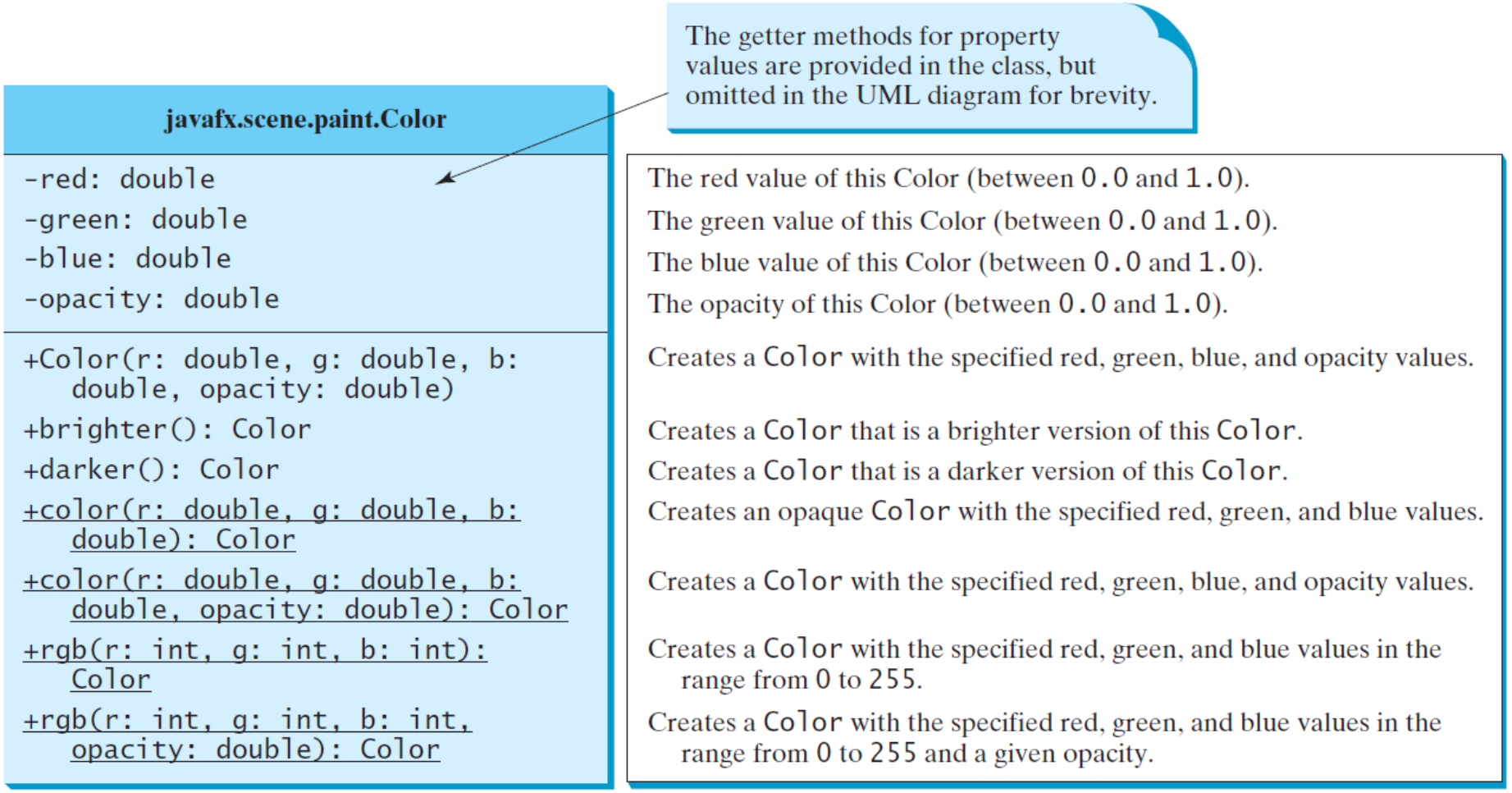
```
import javafx.application.Application;
import javafx.stage.Stage;
import javafx.scene.Scene;
import javafx.scene.layout.Pane;
import javafx.scene.paint.Color;
import javafx.geometry.Insets;
import javafx.scene.text.Text;
import javafx.scene.text.Font;
import javafx.scene.text.FontWeight;
import javafx.scene.text.FontPosture;
public class ShowText extends Application {
  @Override
  public void start(Stage primaryStage) {
    Pane pane = new Pane();
    pane.setPadding(new Insets(5, 5, 5, 5));
    Text text1 = new Text(20, 20, "Programming is fun");
    text1.setFont(Font.font("Courier", FontWeight.BOLD,
      FontPosture.ITALIC, 15));
    pane.getChildren().add(text1);
    Text text2 = new Text(60, 60, "Programming is fun\nDisplay text");
    pane.getChildren().add(text2);
    Text text3 = new Text(10, 100, "Programming is fun\nDisplay text");
    text3.setFill(Color.RED);
    text3.setUnderline(true);
    text3.setStrikethrough(true);
    pane.getChildren().add(text3);
    Scene scene = new Scene(pane, 600, 800);
    primaryStage.setScene(scene); primaryStage.show();
  }
...
}
```





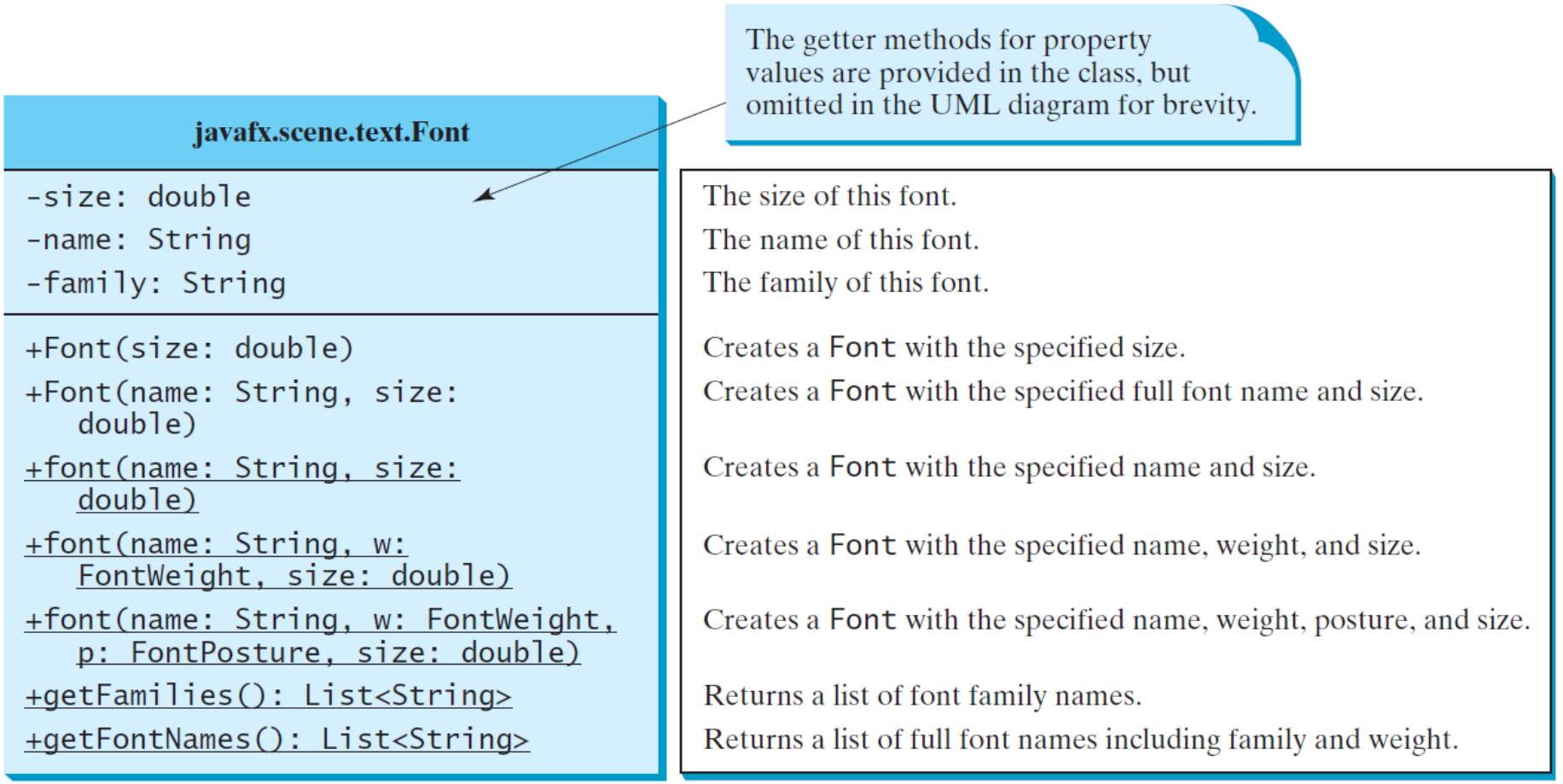
(c) Paul Fodor and Pearson Inc.

Helper classes: The Color Class



(c) Paul Fodor and Pearson Inc.

Helper classes: The Font Class



(c) Paul Fodor and Pearson Inc.

Line

The getter and setter methods for property values and a getter for property itself are provided in the class, but omitted in the UML diagram for brevity.

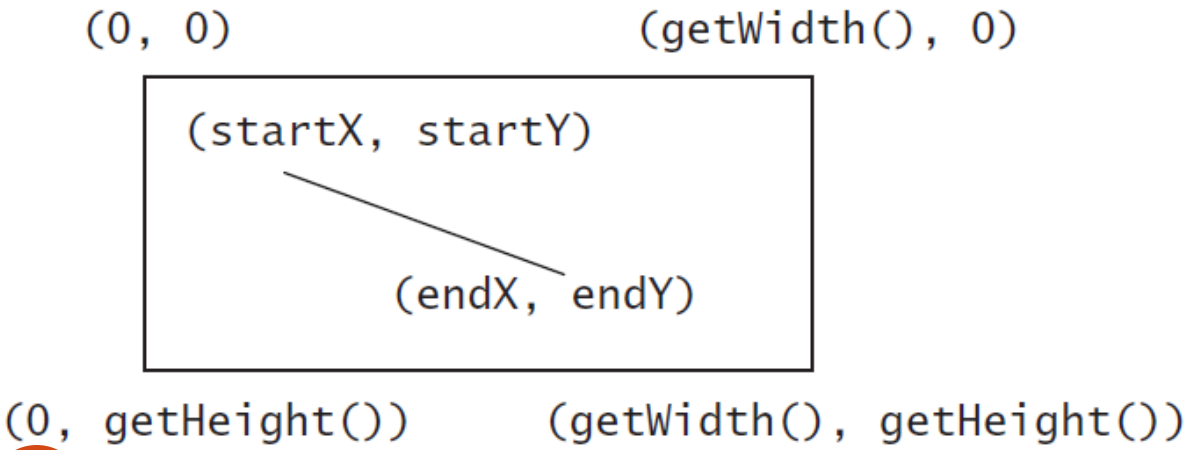
javafx.scene.shape.Line

```
-startX: DoubleProperty
-startY: DoubleProperty
-endX: DoubleProperty
-endY: DoubleProperty
```

```
+Line()
+Line(startX: double, startY:
   double, endX: double, endY:
   double)
```

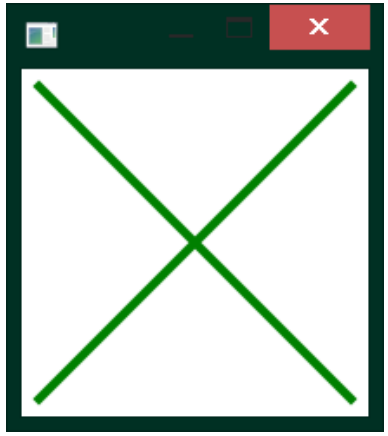
The x-coordinate of the start point.
The y-coordinate of the start point.
The x-coordinate of the end point.
The y-coordinate of the end point.

Creates an empty `Line`.
Creates a `Line` with the specified starting and ending points.



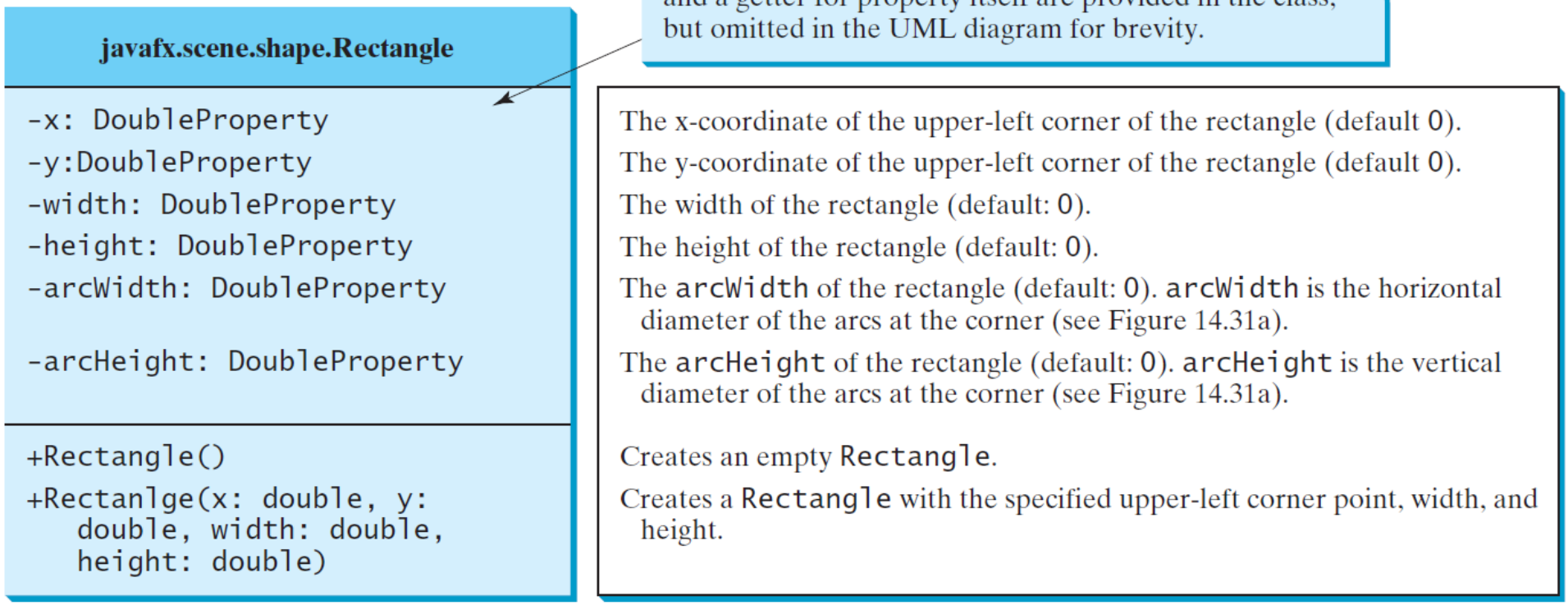
(c) Paul Fodor and Pearson Inc.

```
import javafx.application.Application;
import javafx.stage.Stage;
import javafx.scene.Scene;
import javafx.scene.layout.Pane;
import javafx.scene.shape.Line;
import javafx.scene.paint.Color;
public class ShowLine extends Application {
    @Override
    public void start(Stage primaryStage) {
        Pane pane = new Pane();
        Line line1 = new Line(10, 10, 10, 10);
        line1.endXProperty().bind(pane.widthProperty().subtract(10));
        line1.endYProperty().bind(pane.heightProperty().subtract(10));
        line1.setStrokeWidth(5);
        line1.setStroke(Color.GREEN);
        pane.getChildren().add(line1);
        Line line2 = new Line(10, 10, 10, 10);
        line2.startXProperty().bind(pane.widthProperty().subtract(10));
        line2.endYProperty().bind(pane.heightProperty().subtract(10));
        line2.setStrokeWidth(5);
        line2.setStroke(Color.GREEN);
        pane.getChildren().add(line2);
        Scene scene = new Scene(pane, 200, 200);
        primaryStage.setScene(scene);
        primaryStage.show();
    }
    public static void main(String[] args) {
        launch(args);
    }
}
```



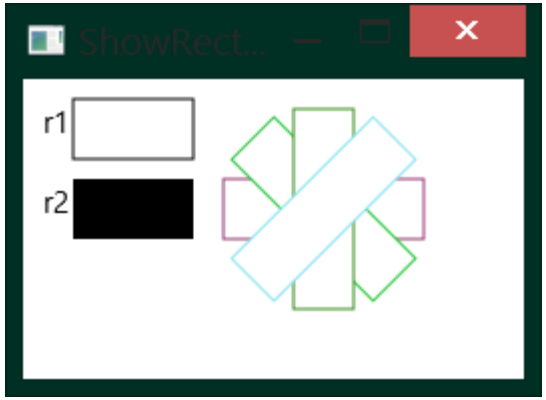
(c) Paul Fodor and Pearson Inc.

Rectangle



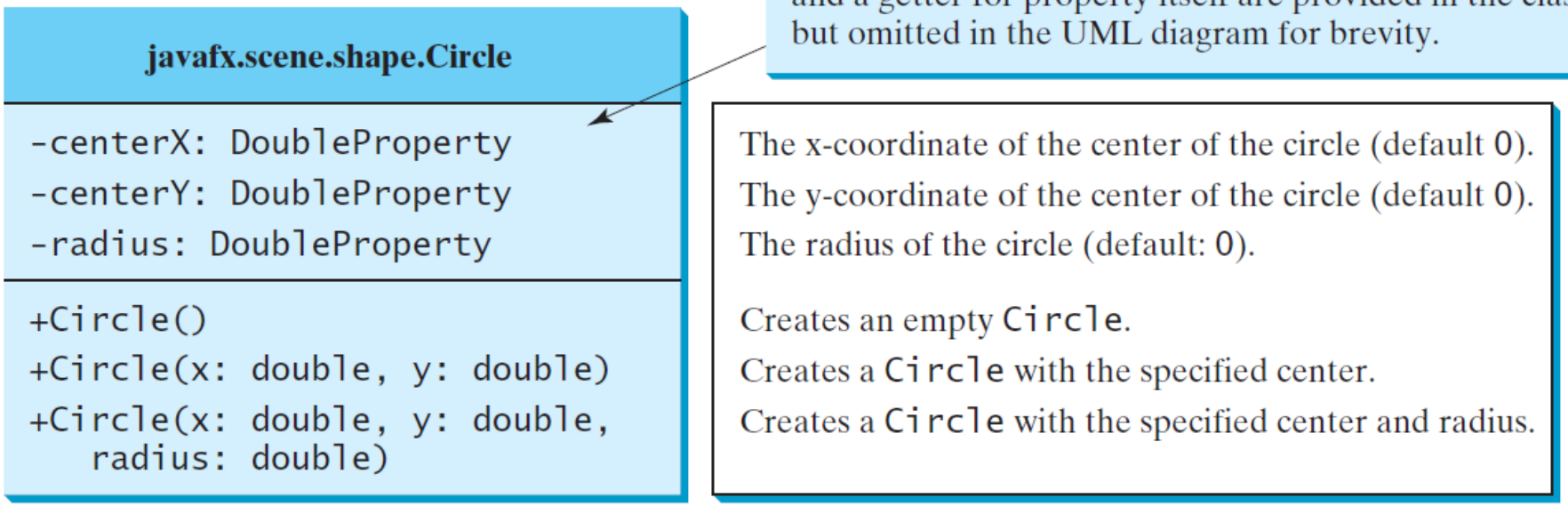
(c) Paul Fodor and Pearson Inc.

```
import javafx.application.Application;
import javafx.stage.Stage;
import javafx.scene.Scene;
import javafx.scene.layout.Pane;
import javafx.scene.text.Text;
import javafx.scene.shape.Rectangle;
import javafx.scene.paint.Color;
import java.util.Collections;
public class ShowRectangle extends Application {
  public void start(Stage primaryStage) {
    Pane pane = new Pane();
    Rectangle r1 = new Rectangle(25, 10, 60, 30);
    r1.setStroke(Color.BLACK);
    r1.setFill(Color.WHITE);
    pane.getChildren().add(new Text(10, 27, "r1"));
    pane.getChildren().add(r1);
    Rectangle r2 = new Rectangle(25, 50, 60, 30);
    pane.getChildren().add(new Text(10, 67, "r2"));
    pane.getChildren().add(r2);
    for (int i = 0; i < 4; i++) {
      Rectangle r = new Rectangle(100, 50, 100, 30);
      r.setRotate(i * 360 / 8);
      r.setStroke(Color.color(Math.random(), Math.random(),
          Math.random()));
      r.setFill(Color.WHITE);
      pane.getChildren().add(r);
    }
    Scene scene = new Scene(pane, 250, 150);
    primaryStage.setScene(scene); primaryStage.show();
  }
  ...// main
```



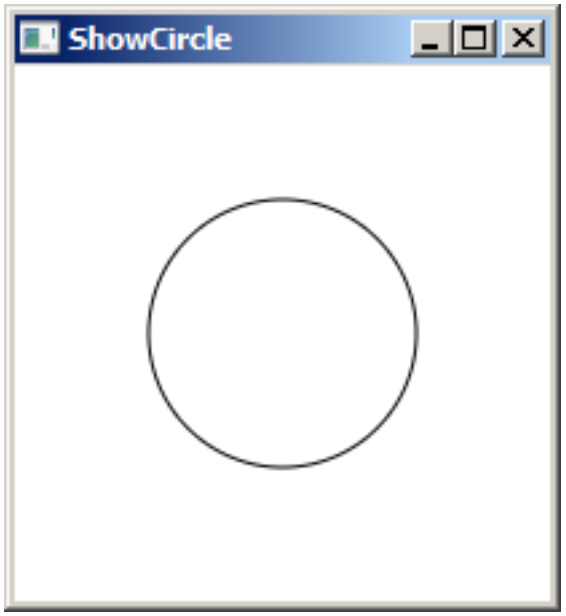
(c) Paul Fodor and Pearson Inc.

Circle



(c) Paul Fodor and Pearson Inc.

Circle in a Pane




```
import javafx.application.Application;
import javafx.stage.Stage;
import javafx.scene.Scene;
import javafx.scene.layout.Pane;
import javafx.scene.shape.Circle;
import javafx.scene.paint.Color;

public class ShowCircle extends Application {
  @Override // Override the start method in the Application class
  public void start(Stage primaryStage) {
    // Create a circle and set its properties
    Circle circle = new Circle();
    circle.setCenterX(100);
    circle.setCenterY(100);
    circle.setRadius(50);
    circle.setStroke(Color.BLACK);
    circle.setFill(null);
    // Create a pane to hold the circle
    Pane pane = new Pane();
    pane.getChildren().add(circle);
    // Create a scene and place it in the stage
    Scene scene = new Scene(pane, 200, 200);
    primaryStage.setTitle("ShowCircle"); // Set the stage title
    primaryStage.setScene(scene); // Place the scene in the stage
    primaryStage.show(); // Display the stage
  }

  public static void main(String[] args) {
    launch(args);
  }
}
```


(c) Paul Fodor and Pearson Inc.

Ellipse

The getter and setter methods for property values and a getter for property itself are provided in the class, but omitted in the UML diagram for brevity.

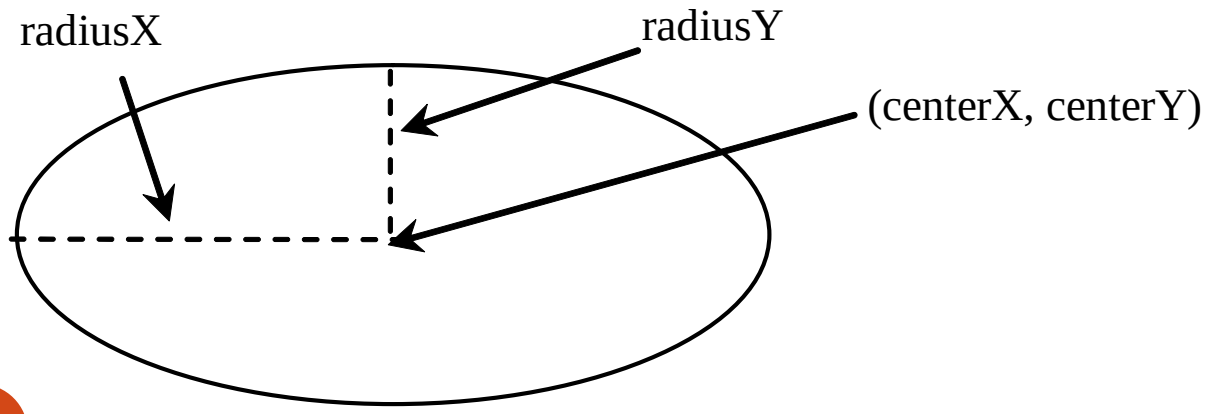
javafx.scene.shape.Ellipse

```
-centerX: DoubleProperty
-centerY: DoubleProperty
-radiusX: DoubleProperty
-radiusY: DoubleProperty
```

```
+Ellipse()
+Ellipse(x: double, y: double)
+Ellipse(x: double, y: double,
   radiusX: double, radiusY:
   double)
```

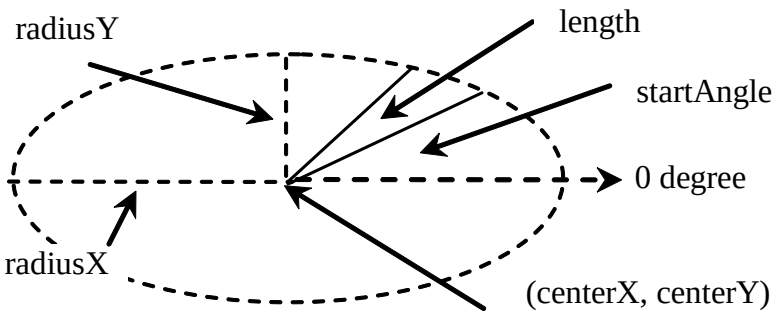
The x-coordinate of the center of the ellipse (default 0).
The y-coordinate of the center of the ellipse (default 0).
The horizontal radius of the ellipse (default: 0).
The vertical radius of the ellipse (default: 0).

Creates an empty `Ellipse`.
Creates an `Ellipse` with the specified center.
Creates an `Ellipse` with the specified center and radiuses.



(c) Paul Fodor and Pearson Inc.

Arc



The getter and setter methods for property values and a getter for property itself are provided in the class, but omitted in the UML diagram for brevity.

javafx.scene.shape.Arc

```
-centerX: DoubleProperty
-centerY: DoubleProperty
-radiusX: DoubleProperty
-radiusY: DoubleProperty
-startAngle: DoubleProperty
-length: DoubleProperty
-type: ObjectProperty<ArcType>
```

```
+Arc()
+Arc(x: double, y: double,
   radiusX: double, radiusY:
   double, startAngle: double,
   length: double)
```

The x-coordinate of the center of the ellipse (default 0).

The y-coordinate of the center of the ellipse (default 0).

The horizontal radius of the ellipse (default: 0).

The vertical radius of the ellipse (default: 0).

The start angle of the arc in degrees.

The angular extent of the arc in degrees.

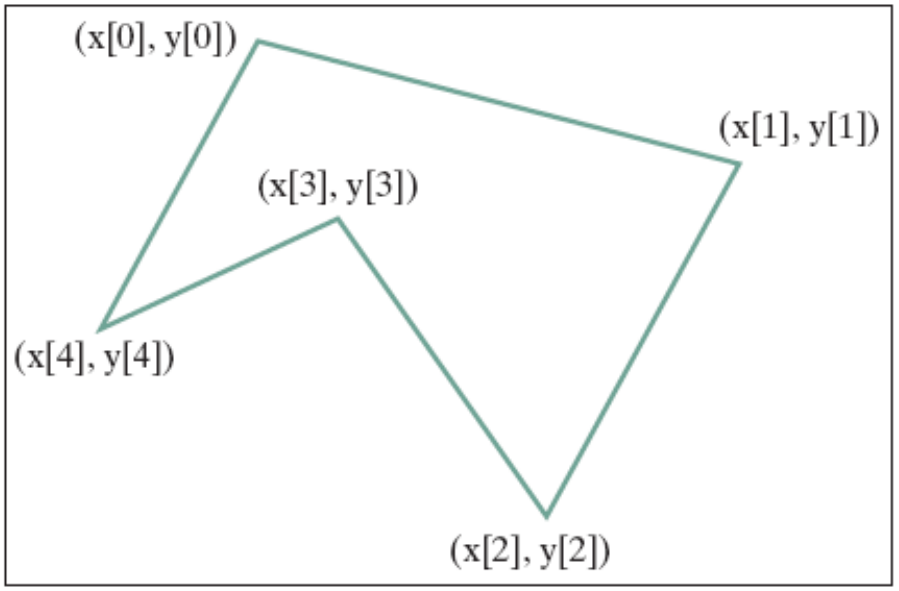
The closure type of the arc (`ArcType.OPEN`, `ArcType.CHORD`, `ArcType.ROUND`).

Creates an empty `Arc`.

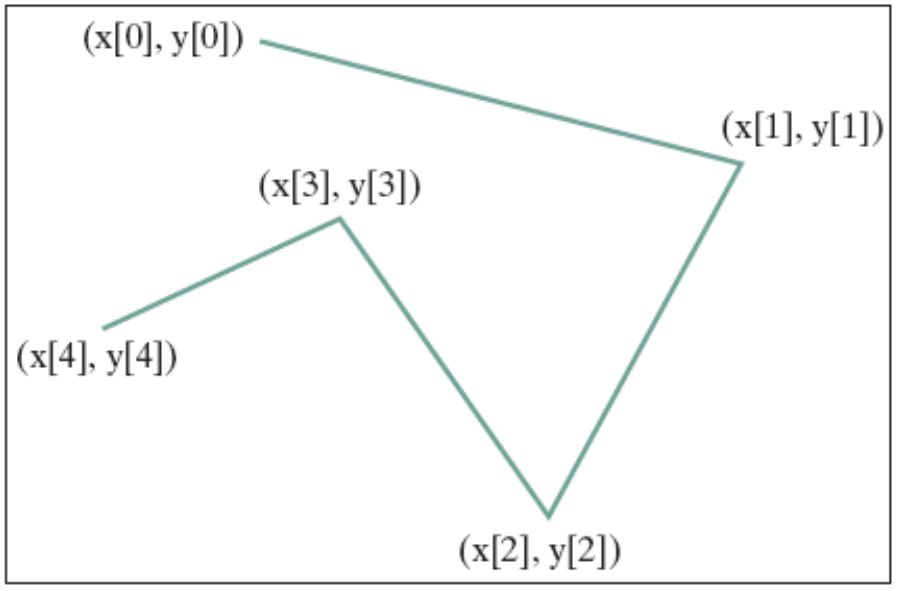
Creates an `Arc` with the specified arguments.

(c) Paul Fodor and Pearson Inc.

Polygon and Polyline



(a) Polygon



(b) Polyline

javafx.scene.shape.Polygon
`+Polygon()`
`+Polygon(double... points)`
`+getPoints(): ObservableList<Double>`

The `getter` and `setter` methods for property values and a getter for property itself are provided in the class, but omitted in the UML diagram for brevity.

Creates an empty polygon.

Creates a polygon with the given points.

Returns a list of double values as x- and y-coordinates of the points.

(c) Paul Fodor and Pearson Inc.

The Image and ImageView Classes

The getter methods for property values are provided in the class, but omitted in the UML diagram for brevity.

javafx.scene.image.Image

```
-error: ReadOnlyBooleanProperty
-height: ReadOnlyBooleanProperty
-width: ReadOnlyBooleanProperty
-progress: ReadOnlyBooleanProperty
+Image(filenameOrURL: String)
```

Indicates whether the image is loaded correctly?
The height of the image.
The width of the image.
The approximate percentage of image's loading that is completed.

Creates an `Image` with contents loaded from a file or a URL.

The getter and setter methods for property values and a getter for property itself are provided in the class, but omitted in the UML diagram for brevity.

javafx.scene.image.ImageView

```
-fitHeight: DoubleProperty
-fitWidth: DoubleProperty
-x: DoubleProperty
-y: DoubleProperty
-image: ObjectProperty<Image>
+ImageView()
+ImageView(image: Image)
+ImageView(filenameOrURL: String)
```

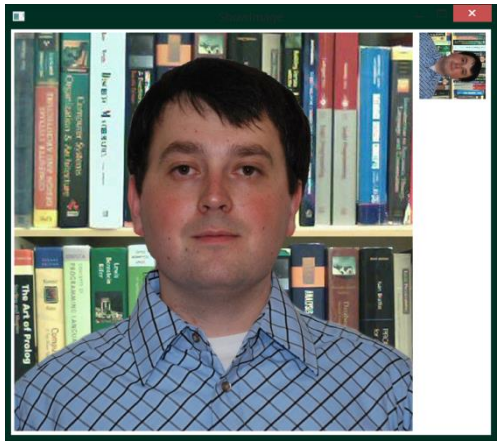
The height of the bounding box within which the image is resized to fit.
The width of the bounding box within which the image is resized to fit.
The x-coordinate of the ImageView origin.
The y-coordinate of the ImageView origin.
The image to be displayed in the image view.

Creates an `ImageView`.
Creates an `ImageView` with the specified image.
Creates an `ImageView` with image loaded from the specified file or URL.

(c) Paul Fodor and Pearson Inc.

```
import javafx.application.Application;
import javafx.stage.Stage;
import javafx.scene.Scene;
import javafx.scene.layout.Pane;
import javafx.scene.layout.HBox;
import javafx.scene.image.Image;
import javafx.scene.image.ImageView;
import javafx.geometry.Insets;

public class ShowImage extends Application {
  @Override
  public void start(Stage primaryStage) {
    // Create a pane to hold the image views
    Pane pane = new HBox(10);
    pane.setPadding(new Insets(5, 5, 5, 5));
    Image image = new Image("paul.jpg");
    pane.getChildren().add(new ImageView(image));
    ImageView imageView2 = new ImageView(image);
    imageView2.setFitHeight(100);
    imageView2.setFitWidth(100);
    imageView2.setRotate(90);
    pane.getChildren().add(imageView2);
    Scene scene = new Scene(pane);
    primaryStage.setTitle("ShowImage");
    primaryStage.setScene(scene);
    primaryStage.show();
  }
  public static void main(String[] args) {
    launch(args);
  }
}
```

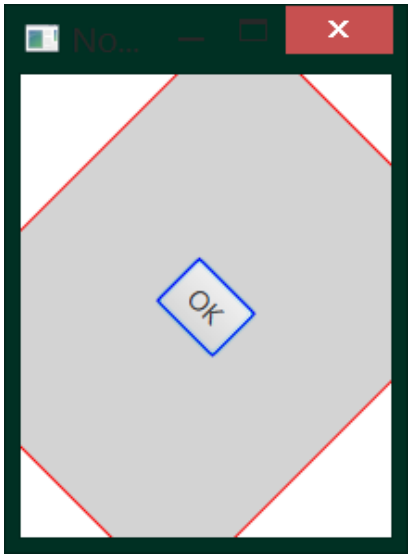


(c) Paul Fodor and Pearson Inc.

JavaFX CSS style and Node rotation

```
import javafx.application.Application;
import javafx.stage.Stage;
import javafx.scene.Scene;
import javafx.scene.layout.StackPane;
import javafx.scene.control.Button;
public class NodeStyleRotateDemo extends Application {
  @Override
  public void start(Stage primaryStage) {
    StackPane pane = new StackPane();
    Button btOK = new Button("OK");
    btOK.setStyle("-fx-border-color: blue;");
    pane.getChildren().add(btOK);
    pane.setRotate(45);
    pane.setStyle("-fx-border-color: red; -fx-background-color: lightgray;");
    Scene scene = new Scene(pane, 200, 250);
    primaryStage.setTitle("NodeStyleRotateDemo"); // Set the stage title
    primaryStage.setScene(scene); // Place the scene in the stage
    primaryStage.show(); // Display the stage
  }

  public static void main(String[] args) {
    launch(args);
  }
}
```

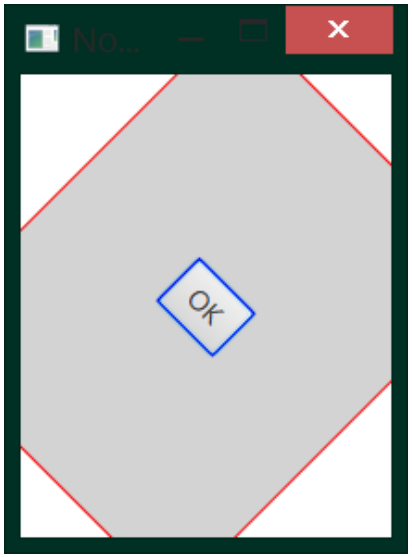


(c) Paul Fodor and Pearson Inc.

JavaFX CSS style and Node rotation

```
import javafx.application.Application;
import javafx.stage.Stage;
import javafx.scene.Scene;
import javafx.scene.layout.StackPane;
import javafx.scene.control.Button;

public class NodeStyleRotateDemo extends Application {
  @Override
  public void start(Stage primaryStage) {
    StackPane pane = new StackPane();
```




```
/* The StackPane layout pane places all of the nodes within
a single stack with each new node added on top of the
previous node. This layout model provides an easy way to
overlay text on a shape or image and to overlap common
shapes to create a complex shape. */
```


(c) Paul Fodor and Pearson Inc.

JavaFX External CSS style file

```
// Example to load and use a CSS style file in a scene
import javafx.application.Application;
import javafx.stage.Stage;
import javafx.scene.Scene;
import javafx.scene.layout.BorderPane;

public class ExternalCSSFile extends Application {
  @Override
  public void start(Stage primaryStage) {
    try {
      BorderPane root = new BorderPane();
      Scene scene = new Scene(root,400,400);
      scene.getStylesheets().add(getClass()
        .getResource("application.css").toExternalForm());
      primaryStage.setScene(scene);
      primaryStage.show();
    } catch(Exception e) {
      e.printStackTrace();
    }
  }
  public static void main(String[] args) {
    launch(args);
  }
}
```


(c) Paul Fodor and Pearson Inc.